Prisma 란?

🧭 ORM이란?

- **Object-Relational Mapping**
- 객체와 관계형 데이터베이스(DB)를 매핑
- SQL 없이 자바스크립트/타입스크립트 코드로 DB 조작 가능

```
-- 직접 SQL 사용 시
const user = await this.dataSource.query(
  `SELECT * FROM users WHERE email = ?`,
  ['example@test.com']
);
```

```
// ORM 사용 시
const user = await this.userRepository.findOne({
  where: { email: 'example@test.com' },
});
```

> 📌 가독성, 재사용성, 생산성 증가

🚀 Prisma란?

- Node.js & TypeScript 환경에서 사용하는 **현대적인 ORM**
- 특징:
  - **타입 안전성**
  - **자동완성 지원**
  - **마이그레이션 관리**
  - **간단한 문법**

> 초기엔 성능이 제한적이었지만 최근 빠른 발전과 투자로 빠르게 발전 중

🔥 Prisma의 인기

- GitHub Stars 기준 **Node.js ORM 1위**
- 실제 개발자들 사이에서 **만족도 매우 높음**
- NestJS와 함께 쓸 때 **시너지 극대화**
  - 복잡한 JOIN, 서브쿼리도 간결하게 처리

🧱 Prisma 구성: Schema & Client

🔷 1. Prisma Schema (`schema.prisma`)

- DB 모델과 관계 정의
- Prisma 프로젝트의 설계도 역할

```
model User {
  id    Int    @id @default(autoincrement())
  email String @unique
  name  String?
  posts Post[]
}
```

> Schema 작성 → 마이그레이션 수행 → DB 테이블 생성

🔷 2. Prisma Client

- Schema 기반으로 자동 생성되는 **DB 접근 객체**
- **자동 타입 지원** → 실수 방지 + 생산성 향상

```
const user = await this.prisma.user.findUnique({
  where: { email: 'test@example.com' },
});
```

> 📌 어떤 필드를 사용할 수 있는지 자동 완성으로 안내

✅ 정리

- ORM은 SQL 없이 DB를 다루는 도구
- Prisma는 **타입 안정성과 개발자 경험(DevEx)**이 뛰어난 최신 ORM
- Schema와 Client를 기반으로 구성됨
- **실습 전에 이 개념만 간단히 이해하고 넘어가면 충분**